

AI 시대 플랫폼 팀의 역할과 핵심 역량

개발자 경험을 재정의하고, 조직의 지능을 설계하는 새로운 플랫폼 전략



📅 2025-10-30

👤 Vizend Workshop

목차



1. AI 시대, 왜 플랫폼 팀인가?



2. 플랫폼 팀의 역할 변화



3. 개발자 경험 향상



4. 글로벌 기업 사례



5. 플랫폼 팀에 필요한 핵심 역량



6. 플랫폼 엔지니어 인재상



7. 결론

WHY

플랫폼의 의미와 가치

WHAT & HOW

운영모델과 전략

WHO

필요 역량과 팀 구성

복잡성 폭증과 인지 부하

복잡성의 원인



마이크로서비스

서비스 간 의존성과 통합 관리의 복잡성



클라우드

다양한 서비스와 제공업체 관리의 부담



AI 기술 확산

신기술 학습과 도입의 추가적 노력

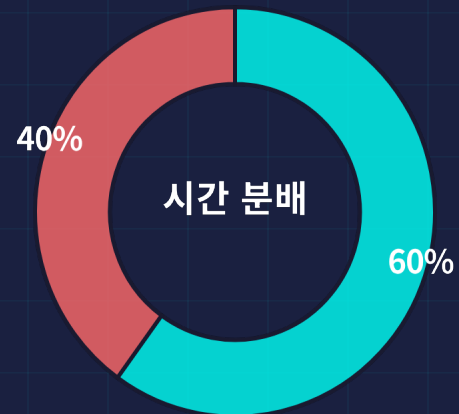
결과: 개발 생산성 저해, 혁신 속도 감소

복잡한 인프라와 반복적인 작업에 얽매어 창의적인 문제 해결에 집중하지 못합니다.



Stripe Developer Coefficient 2023

600명의 개발자를 대상으로 조사한 결과



■ 개발 업무: 60%

■ 비개발 업무: 40%

비개발 업무

- 회의와 커뮤니케이션
- 태스크 관리
- 고객지원 / 응대
- 보안 / 컴플라이언스

Time Warp: 개발자의 시간은 어디로 사라지는가

개발자가 실제로 시간을 쓰는 곳



코딩

이상 20% vs 실제: 11%



설계

이상 15% vs 실제: 6%



회의·커뮤니케이션

이상 8% vs 실제: 12%



보안 & 컴플라이언스

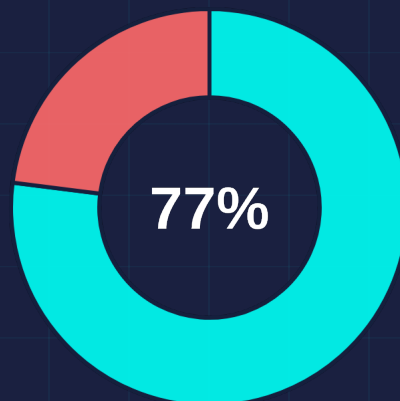
이상 4% vs 실제: 10%

WorkWeek: 이상 vs 현실

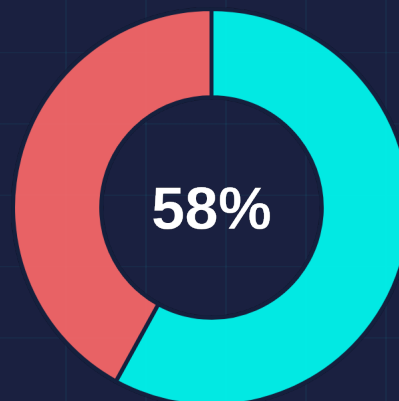


Microsoft 연구 결과

실제 근무 주간과 이상적 근무 주간 비교



이상 근무 주



실제 근무 주

■ 개발 업무

■ 비개발 업무



핵심 인사이트

Microsoft 연구의 결론: 이상 Workweek와 실제 Workweek의 괴리가 클수록 개발자 생산성과 만족도가 모두 하락한다.

<https://www.microsoft.com/en-us/research/wp-content/uploads/2024/11/Time-Warp-Developer-Productivity-Study.pdf>

AI 시대 플랫폼팀의 목표



핵심 결론

AI 시대 플랫폼팀은 단순한 코드 자동화 팀이 아니라 **모든 개발자의 '시간을 효율화하는 팀'**이 되어야 한다.

핵심 목표 3가지



이상적인 Workweek에 가까운 구조 설계
→ 비개발 업무 자동화, 환경 표준화



개발자 경험 중심의 셀프서비스 플랫폼 구축
→ 팀 간 의존 최소화, 개발자 자율성 극대화



AI + 플랫폼팀 시너지 강화
→ 문서·테스트·보안·커뮤니케이션 자동화

핵심 성과 지표

지표	정의	기대 효과
Time Recovery Rate (TRR)	개발자가 되찾은 집중 업무 시간 비율	생산성·몰입도 향상
Automation Coverage	자동화된 비개발 업무 비중	운영 효율 증대
개발자 경험 만족도	내부 개발자 만족도	유지율 및 품질 향상

시간은 개발자의 가장 비싼 리소스다.
플랫폼 팀은 개발자의 잃어버린 시간을 되찾아줘야 한다.

플랫폼팀의 진화: Supporter → Accelerator → OS



과거: Supporter

역할 정의

인프라 안정·표준을 지키는 운영팀

조직에 주는 가치

개발팀 생산성 **개별** 향상

특징

- ✓ 모놀리식 아키텍처, 온프레미스 운영
- ✓ 서버, 네트워크, 데이터베이스 관리
- ✓ 티켓 기반의 요청 처리 및 문제 해결



현재: Accelerator

역할 정의

개발자 경험과 협업을 설계하는 전략팀

조직에 주는 가치

조직 속도와 품질 **전사** 향상

특징

- ✓ 클라우드, 마이크로서비스, DevOps
- ✓ 경험 설계: 개발자의 전체 여정을 디자인
- ✓ 팀 간의 사일로를 허물고 협업 활성화



AI 시대: 조직의 OS

역할 정의

AI Native Enabler, 문화 설계자

조직에 주는 가치

비즈니스 경쟁력 결정

특징

- ✓ AI 네이티브 시대, 조직의 운영체제
- ✓ 새로운 기술과 아이디어를 실험하고 확산
- ✓ 안정성과 혁신의 균형

플랫폼 경험의 두 축

Developer Experience

×

Developer Relationship

=

Innovation Speed



Developer Experience (DX)

- 개발자의 전체 여정을 디자인하고 최적화
- 자동화된 워크플로와 표준 템플릿 제공
- 복잡한 인프라나 반복적인 작업에 얽매이지 않도록 돕기
- 개발자 생산성과 만족도 향상



Developer Relationship (DevRel)

- 개발자 간 협력과 신뢰 강화
- 지식 공유와 혁신 촉진
- 팀 간의 사일로를 허물고 소통 활성화
- 지속 가능한 거버넌스와 안전한 가이드레일 설계



핵심 인사이트

기술적 우수성(DX)과 개발자 관계(DR)가 결합될 때 조직의 혁신 속도가 결정됩니다. 플랫폼 팀의 역할은 이 두 축을 설계하고 증폭시키는 것입니다.

내부 개발자 플랫폼 – Internal Developer Platform

정의

IDP는 개발자가 셀프서비스 방식으로 필요한 리소스를 즉시 확보하고, 지식과 협업이 한 곳으로 모이는 **중앙 포털**입니다. 이는 개발자가 인프라를 의식하지 않고 일하게 만드는 내부 허브입니다.



코드 작성

템플릿/스타트키트



인프라

서비스/환경



IDP

중앙 허브



문서/가이드

학습/참조



커뮤니티

질의/답변

DevEx 기여



통합 UI

한눈에 필요한 모든 도구



자동화된 워크플로

복잡한 프로세스 단순화



표준 템플릿

일관성과 품질 보장



DevRel 기여



서비스 카탈로그

서비스와 API 투명성



문서 및 가이드

표준화된 정보 접근



커뮤니티 기능

지식 공유 활성화

"IDP는 개발자의 인프라 의식을 제거 하는 역할을 합니다"

사례: Spotify Backstage

"모든 인프라를 하나의 인터페이스로" 제공하는 오픈소스 내부 개발자 플랫폼



Service Catalog

모든 팀의 서비스와 API를 투명하게 관리하고 의존성을 시각화하여 팀 간의 이해도를 높입니다.



Software Templates

표준화된 템플릿을 통해 신규 프로젝트를 빠르게 생성하고, 필요한 표준 설정을 자동으로 적용합니다.



TechDocs

코드와 문서의 동기화를 유지하고 실시간 협업을 지원하여 문서화의 효율성을 극대화합니다.

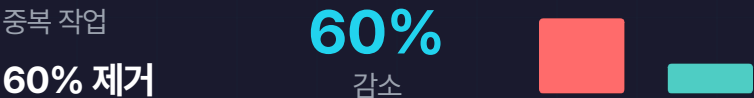
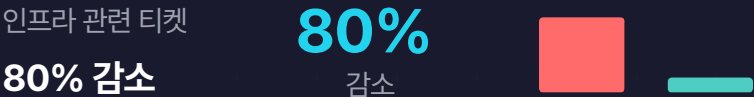


Plugin Ecosystem

500개 이상의 플러그인을 통해 다양한 개발 도구를 통합하여 개발 환경의 일관성을 제공합니다.

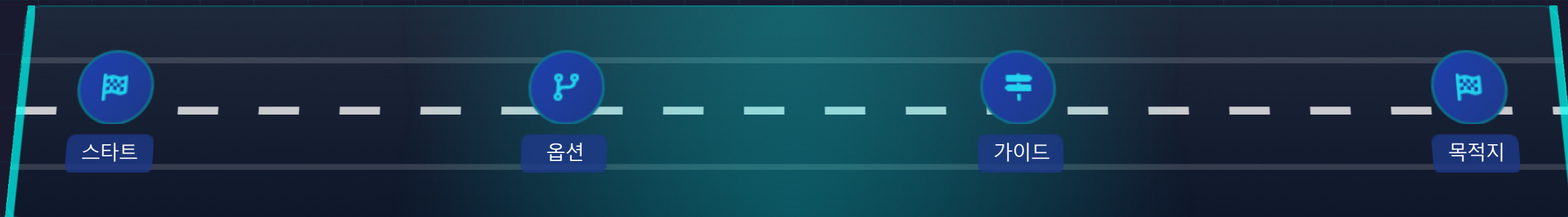
"Backstage는 Spotify의 셀프서비스 개발 플랫폼으로, 모든 팀의 서비스와 API를 한눈에 볼 수 있는 중앙 허브를 제공합니다. 이를 통해 개발자들은 '누가 무엇을 하는지'를 쉽게 파악할 수 있습니다."

정량 효과



핵심 인사이트: Backstage는 플랫폼 팀의 역할이 DevOps를 넘어 DevEx로 향하는 진화를 보여준다.

골든 패스 — 포장된 고속도로



골든 패스(Golden Path)는 대부분의 개발 팀이 빠르고 안전하게 소프트웨어를 개발하고 배포할 수 있도록 미리 포장된 표준 경로입니다. 이는 개발자가 최적의 경로를 쉽게 선택하고 따를 수 있도록 가이드라인과 도구를 제공합니다.

Opinionated: 명확한 기준

팀이 선택 시간을 단축하고 일관성을 유지하도록 명확한 기준과 권장 사항을 제공합니다.

Flexible: 벗어날 수 있는 유연함

필요한 경우 표준에서 벗어날 수 있는 유연성을 제공합니다. 단, 이유와 문서화가 필요합니다.

80/20 Rule: 주요 시나리오에 집중

주요 사용 사례와 작업 흐름에 최적화된 경로를 우선으로 합니다. 특수한 케이스는 따로 처리합니다.

Opt-in: 강제가 아닌 권장

패스를 강제하지 않고 개발자가 의식적으로 채택하도록 합니다. 이로 인해 채택률과 지속성을 높입니다.

AI & 협업 도구 — 통합 전략 맵

AI 생산성 도구와 협업 도구의 통합은 개발자 경험을 향상시키고 조직 혁신을 가속화합니다.

코드 작성

AI 도구

- ✓ GitHub Copilot: 코드 자동 완성
- ✓ Cursor: 리뷰 품질 향상

기대 효과

코드 작성 속도 **+55%**,
리뷰 시간 **62%** 감소

문서화

AI 도구

- ✓ AI Doc Generator: 빠른 문서 생성
- ✓ FAQ 챗봇: 자동화된 문제 해결

기대 효과

문서 작성 시간 **75%** 감소
지식 공유 속도 향상

커뮤니케이션

AI 도구

- ✓ Slack/Telegram Bot: 요약 생성
- ✓ 번역 및 FAQ 자동응답

기대 효과

팀 간 소통 빈도 **3배** 증가
의사결정 속도 **50%** 단축

온보딩

AI 도구

- ✓ AI 온보딩 어시스턴트
- ✓ 실시간 피드백 시스템

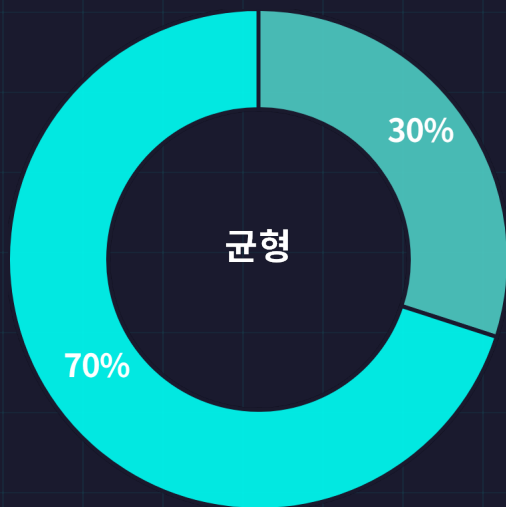
기대 효과

Ramp-Up 기간 **30%** 단축
교육 리소스 절감

 **핵심 인사이트:** AI와 협업 도구의 통합은 개발자가 **핵심 업무에 집중**할 수 있게 합니다. 이는 **조직의 생산성과 만족도를 극대화**하는 데 결정적인 역할을 합니다.

플랫폼 엔지니어 역량의 두 축

기술적 역량은 플랫폼의 신뢰를, 비기술적 역량은 플랫폼의 채택을 만듭니다.



플랫폼 팀의 성공은 균형에 달려 있습니다

기술은 플랫폼을 작동 하게 하고, 비기술 역량은 사용하게 합니다



기술적 역량 (30%)

플랫폼의 신뢰성을 구축하는 기술적 기반입니다.



클라우드/IaaS



CI/CD



신뢰성 & 보안



자동화 & AI



비기술적 역량 (70%)

플랫폼 팀에 대한 신뢰를 구축하는 관계적 기능입니다.



커뮤니케이션



제품 마인드셋



문서화 & 교육



개발자 공감

기술적 역량 - 플랫폼을 신뢰하게 만드는 기반

기술적 역량은 플랫폼의 신뢰를, 비기술적 역량은 플랫폼의 채택을 만든다.



인프라 & 배포

- ✓ IaC: 환경 일관성 확보, 재현 가능한 인프라
- ✓ CI/CD: 개발자가 로직에 집중할 수 있게
- ✓ Self-Service: 개발 속도와 자율성 강화

안정적인 기반으로 신뢰 구축



신뢰성 & 보안

- ✓ Observability: 실시간 가시성 및 안정성 확보
- ✓ MTTR: 장애 감지 대응 복구 속도 최적화
- ✓ 선제적 대응: 사후대응이 아닌 사전설계로

신뢰는 성능이 아니라 회복력에서



자동화 & AI

- ✓ 반복 작업 제거: 개발자의 인지 부하 감소
- ✓ LLM 내재화: AI 기반 도구 안전하게 제공
- ✓ Dev-Assistance: 개발자 경험 지원형 AI

"AI는 개발자의 인지 부하와 운영 비용을 절약"

핵심: 플랫폼의 신뢰는 문제가 없는 시스템이 아니라 문제를 빠르게 감지하고, 복구하고, 학습하는 구조에서 만들어진다.

개발자의 생산성을 결정 짓는 요인



Google 연구 결과

"What Predicts Software Developers' Productivity?" (622명 개발자 조사)

★ 생산성 예측 요인 Top 3

1. Job Enthusiasm

비기술적 요인

자신의 직무에 대해 **긍정적 감정**과 **몰입**을 느끼는 정도

2. Peer Support for New Ideas

비기술적 요인

팀 내/동료가 새로운 아이디어를 받아들이고 **지지해 주는 분위기**

3. Useful Feedback about Job Performance

비기술적 요인

자신의 업무 수행에 대해 **실질적이고 건설적인 피드백**을 받는 정도

상위 10개 항목이 모두 비기술적 요인

코드 품질, 플랫폼, 복잡성 등 (COCOMO II)

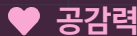
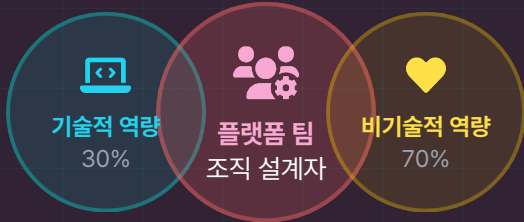
기술적 요인

생산성과의 상관관계가 매우 낮거나 통계적으로 무의미



플랫폼 팀의 역할

단순히 도구를 제공하는 기술 조직이 아닌,
개발자의 동기부여·협업을 증폭시키는 **조직 설계자(Experience Designer)**



공감력



소통 능력



문화 구축



피드백 설계

이것이 조직 전체 생산성의 레버리지 포인트

비기술 역량 - 1. 커뮤니케이션

"권한 없이 설득하고 연결하는 능력"

플랫폼 팀은 직접적인 권한이 없더라도 조직 내외부의 다양한 이해관계자들을 설득하고 연결하는 'Influence without Authority' 능력이 필요합니다.

💡 핵심 메시지

👥 플랫폼 팀은 명령할 권한은 없지만, 설득할 책임은 있다.

💬 커뮤니케이션은 정보 전달이 아니라 의도 해석의 기술

🗣️ 기술의 언어로 시작해, 사람의 언어로 마무리할 수 있는
조직 내 통역사 역량이 필요

Bridge between Tech and Business

🏠 3D 커뮤니케이션 모델

축	의미	예시 행동
❤️ Depth	메시지의 이해도	개발자의 맥락과 Pain Point를 공감적으로 이해
➕ Direction	3차원 커뮤니케이션	경영진↔실무자↔외부 파트너 간 언어 톤 맞춤
📊 Data	데이터 기반 설득	AI 지표, DORA 데이터, DevEx 피드백 활용

커뮤니케이션은 말이 아니라 구조다 - 이해의 흐름을 설계하라.

비기술 역량 - 2. 제품 마인드셋

플랫폼 팀은 코드를 제공하는 팀이 아니라, 조직의 사용자 경험을 설계하는 팀입니다.

💡 제품적 사고 (Product Thinking)

- 🔗 플랫폼을 '도구'가 아니라 **서비스**로 인식
- 👥 DevEx, DevRel, AI를 통합한 "내부 제품 생태계"로 인식
- 🧩 문제해결의 초점을 **기능이 아닌 경험**으로
- ❓ 의사결정 기준: "**기술적으로 가능한가?**"
→ "**사용자가 실제로 쓸까?**"

🎯 사용자 중심 (User Obsession)

- 👤 개발자는 플랫폼의 **고객**이다
- 📊 플랫폼 성공의 핵심 지표는 "**채택률(Adoption Rate)**과 **사용자 만족도(eNPS)**"
- ☰ 사용자 중심 플랫폼의 3요소:
 - 사용 편의성 (Usability)
 - 피드백 루프 (Feedback Loop)
 - 데이터 기반 개선 (Usage Analytics)

🔄 지속적인 개선 (Iterative Growth)

- 🔧 '완성'보다 '**지속적 개선**'을 목표로
- 🕒 MVP → Feedback → Improve → Measure → Repeat
- 🧪 실험 가능한 제품 문화 (Experimental Culture)
- 🕒 변화 대응 속도 = 조직 학습 속도

플랫폼 팀의 경쟁력 = 기술적 완성도 × 고객 중심적 사고 × 지속적 개선 문화

비기술 역량 - 3. 문서화 & 교육

AI 시대 플랫폼 팀은 조직의 지식을 **구조화**하고, 학습을 시스템에 **내재화**하며, AI를 통해 그 가치를 **극대화**해야 한다.

Knowledge (문서화)



Beginner to Expert

Getting Started / ADR 등 체계적인 문서 구조화. 신규 입문자부터 시니어까지 일관된 학습 흐름 제공



Living Docs

코드·문서·토론을 연결한 살아있는 지식체계 갱신 흐름을 포함해 **조직의 기억을 보존**



Knowledge Stewardship

문서를 남기는 데 그치지 않고, **함께 돌보며 지식의 품질을 유지**하는 문화 정착.

Enablement (교육 & 내재화)



Community Enablement

테크 세션·DevTalk 등을 통해 경험 공유를 문화화. 학습을 개인의 성장에서 **조직의 자산**으로 확장.



Developer Journey

신규 개발자/팀 전환자를 위한 온보딩 자동화 문서, 실습, 멘토 챗봇을 연결한 **학습 경로 설계**



Learning in Work

업무 과정 자체가 학습이 되도록 만드는 구조. 도구가 아닌 **경험이 학습을 이끌**게 한다.

Intelligence (AI 기반 학습 지원)



Context-Aware Code Review

코드와 문서, ADR을 연결하여 팀 **고유의 규칙·패턴을 학습**한 AI 기반 리뷰



Learning Assistant

개인별 코드·문서·이슈를 학습해 맞춤형 성장 방향을 제안하는 AI 코치.



Knowledge Feedback Loop

팀 내 문서·코드 변경 패턴을 분석해, **학습과 개선의 루프를 자동화**

비기술 역량 – 4. 개발자 공감력

💡 개발자 공감력이란?

개발자 공감력은 단순한 감정적 이해가 아닌 체계적인 통찰력과 실천적인 방법론을 필요로 합니다.



정의

플랫폼 팀이 개발자의 업무 흐름, 어려움, 요구사항을 깊이 이해하고 개발자 관점에서 경험을 설계하는 능력



중요성

공감력은 플랫폼이 선택받게 하는 설득의 힘



인지 부하 설계

공감은 '감정'이 아니라 인지적 부담을 줄이는 구조적 설계 행위

🔧 실천 방법 4가지

🐾 도그 푸딩

플랫폼 팀이 자신의 플랫폼 사용

📖 마찰 로깅

플랫폼 사용시 겪는 모든 불편함 기록

👤 새도잉

개발자 옆에서 하루 관찰

🗨️ 피드백 루프 구축

주기적 개선으로 지속 가능한 신뢰 구축

⚠️ 공감력 부족의 실패 사례

❌ 사례 1: 완벽한 기술, 하지만 배우는 데 2주 필요 → 채택률 감소

❌ 사례 2: 50페이지 문서, 하지만 Getting Started 없음 → 사용 포기

❌ 사례 3: 강력한 기능, 하지만 기존 워크플로우 변경 강요 → 저항

💡 핵심 인사이트: 기술 도입의 성공은 기술 자체의 우수성보다 구성원의 인식에 달려있다. *Min Ren, Why technology adoption succeed or fails*

측정 프레임워크 – DevEx & DevRel 성공 지표



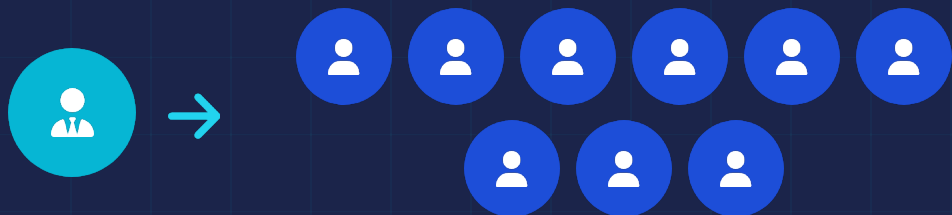
“ 플랫폼팀의 성공은 기술과 감정, 두 언어로 증명된다. ”

Force Multiplier - 1명이 100명을 활성화

💡 공감은 플랫폼 성공의 비밀 무기

플랫폼 엔지니어는 생산성을 증폭시키는 Force Multiplier입니다.
단 한 번의 개선이 수십 명의 개발자 경험을 바꿀 수 있습니다.

플랫폼 엔지니어의 레버리지 효과



10시간 투자 → 주 90시간 절감

한 번의 개선이 수십 명의 개발자 경험을 바꾼다

플랫폼 엔지니어는 **한 팀이 아닌 전체를 바라봐야 한다**

레버리지 사고 4단계

1 패턴 발견

팀별 문제를 넘어 조직 전체의
공통 문제를 식별

2 추상화

공통 문제를 추상화하고,
표준화된 형태로 일반화

3 확산

조직 전체로 솔루션을 확산
새로운 사용 패턴을 촉진

4 측정

효과를 측정하고, 지속적인 개선을 통
해 레버리지 극대화합니다.

핵심 질문

일반 개발자의 사고

"내 팀의 문제를 해결하자"

해결 방식

One-off 스크립트, 수동 대응

플랫폼 엔지니어의 사고

"모든 팀이 겪는 문제인가?"

해결 방식

재사용 가능한 플랫폼/템플릿화 구축

일반 개발자와 플랫폼 엔지니어

일반 개발자

기능을 만드는 사람



고객

외부 사용자 (고객, 최종 사용자)



기여 방식

직접 문제 해결



결과물

코드 / 기능



성공 지표

기능 출시, 사용자 만족, 비즈니스 KPI

VS

핵심 가치

창조(Creation)

VS

증폭(Amplification)



플랫폼 엔지니어

기능을 더 잘 만들 수 있게 하는 사람



고객

내부 개발자 (동료 엔지니어)



기여 방식

문제 해결 방식을 시스템화



결과물

플랫폼 / 도구 / 문화



성공 지표

채택률, 개발자 만족도(eNPS), DORA

"일반 개발자는 건물을 짓는 사람 이고, 플랫폼 엔지니어는 더 나은 도구와 청사진을 제공하는 사람 이다."

역할과 마인드셋 비교

</> 일반 개발자

플랫폼 엔지니어



목표

기능 완성, 출시 속도

조직 전체 생산성·협업 증폭



사고 범위

팀/제품 중심 (마이크로)

전사 경험 (매크로)



가치 측정

사용자 지표, 버그 수 - Output

채택률, eNPS, DORA - Outcome



문제 접근

눈앞의 이슈를 신속히 해결 (Firefighting)

시스템적 근본 제거, 패턴 찾기



커뮤니케이션

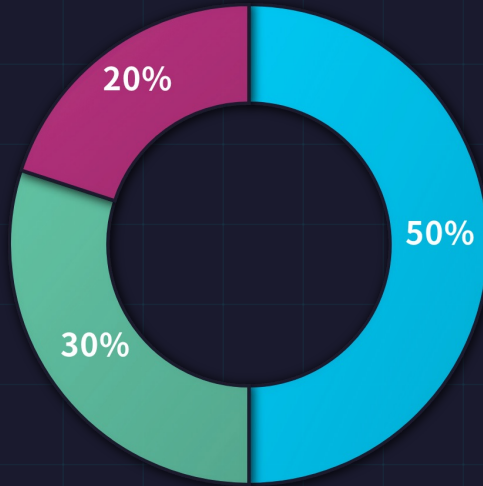
팀 내부 위주

조직 전체+경영진+커뮤니티

일반 개발자가 기능의 '양'을 통해 평가 받는다면, 플랫폼 엔지니어는 생산성의 '파급력'을 통해 평가받는다

플랫폼 엔지니어의 시간 지도

시간 배분



● 기술·플랫폼 구축

기술적 코딩과 플랫폼 개발에 전념

시간 비중 **50%**

● 커뮤니케이션·관계

내부 개발자들과 소통 및 교육

시간 비중 **30%**

● 피드백·지표·개선

플랫폼 효과 측정하고 지속적 개선

시간 비중 **20%**

역할 스펙트럼



엔지니어

코드 중심

- ✓ 기술적 코딩과 문제 해결
- ✓ 플랫폼 아키텍처 설계
- ✓ 자동화 도구 개발



PM

우선순위 & 지표

- ✓ 로드맵 관리
- ✓ 지표 분석 및 보고
- ✓ 중요 문제 우선순위 설정



커뮤니티 리더

소통 & 교육

- ✓ 개발자 채용 및 온보딩
- ✓ 문서화 및 교육
- ✓ 개발자 커뮤니티 활성화



하이브리드 인재

통합적 사고

- ✓ 정량(DORA)과 정성(eNPS) 지표
- ✓ 기술과 사람 사이 간극 메우기
- ✓ 기술적/비기술적 역량 균형

"기술과 사람 사이에서 균형을 잡으며 시간을 배분하는 하이브리드 인재가 플랫폼 팀의 힘을 낼 것입니다."

인재 발굴 우선순위

내부에서 먼저 찾고, 외부는 관점과 전문성을 보완하는 촉매제로 활용

내부 발굴

개발자 경험 개선에 관심 있거나 자발적 도구 개선 이력, 멘토링 활동 활발한 인재 탐색

내부 발굴 시그널:

-  **자발적 개선자**
효율적인 프로세스 제안, 팀 내 도구/스크립트 만들어 공유
-  **멘토/교육자**
신입 온보딩 자발적 도움, 기술 문서 작성, 사내 기술 세미나 발표
-  **문제 제기자 (긍정적)**
불만이 아닌 개선 제안, 다른 팀 문제도 관심
-  **연결자**
팀 간 중재/조율 역할

핵심 인사이트

"완벽한 슈퍼맨을 찾지 말고, 팀으로 역량을 채워라" - 플랫폼 팀은 다양한 역량을 가진 팀원들의 조합으로 성공한다.

외부 채용

조직에 새로운 관점을 줄 수 있으면서 전문성 보완

외부 채용 시 주의사항:

-  **"빅테크 경험"만 보지 마라**
우리 조직 규모/문화와 다름, 문화 적합성 평가
-  **기술만 보지 마라**
플랫폼 = 70% 사람 스킬, 커뮤니케이션 역량 검증
-  **"구원자" 기대 말라**
1명이 조직 바꿀 수 없음, 팀으로 변화, 시간 필요

“ 외부 채용은 특히 조직 적합성과 커뮤니케이션 역량 검증 필수

플랫폼 팀 구성 예시

한 사람의 슈퍼맨이 아니라, 서로 보완되는 소수 정예 팀이 필요합니다.



Tech Lead

- ✓ 클라우드/아키텍처 총괄
- ✓ 가드레일 설계
- ✓ 기술 전략 수립

CI/CD 전문가

- ✓ 자동화/배포 파이프라인
- ✓ 품질 확보
- ✓ 안전한 릴리스 전략

제품 중심 엔지니어

- ✓ 로드맵/지표 관리
- ✓ 사용자 리서치
- ✓ 채택률/만족도 모니터링

커뮤니티 매니저형

- ✓ 문서화/교육
- ✓ 커뮤니케이션 / DevRel
- ✓ 기술 블로그 등 운영

시니어 제너럴리스트

- ✓ 폭넓은 경험으로 브릿지
- ✓ 멘토링/문제 해결 및 조정자
- ✓ 조직 외 전문가 관계

팀 운영 원칙

 T자형 역량: 넓은 기술 스택 + 깊은 전문성 + 비기술적 스킬

 외부 파트너/챗터(보안, SRE, 데이터)와 느슨한 연합

인재 육성 - 4단계 로드맵



1. 기술 역량

- ✓ 클라우드 인증 취득
- ✓ 컨퍼런스 지원 및 참여
- ✓ 기술 전문성 기르기



2. 제품 역량

- ✓ PM 웨도잉
- ✓ 데이터 분석 교육
- ✓ 제품 마인드셋 기르기



3. 소프트 스킬

- ✓ 프레젠테이션 기술
- ✓ 글쓰기 및 스토리텔링
- ✓ 커뮤니케이션 역량 강화



4. 순환 보직

- ✓ 개발팀 파견 (3~6개월)
- ✓ 실제 Squad에서 사용 경험
- ✓ 개발자 공감 체득

💡 채용보다 육성이 더 중요하다

플랫폼 엔지니어는 만들어진다. 기술적인 역량과 비기술적인 역량을 모두 갖춘 인재를 찾아내는 것도 중요하지만, 지속적인 투자를 통해 인재를 성장시키는 것이 더 효과적입니다.

★ 스포티파이 케이스 스터디

Spotify에서 Backstage 팀원들을 실제 Squad에 파견시켜 직접 사용하게 하면서 채택률이 **2배** 증가했습니다. 이는 개발자 경험과 개발자 관계를 동시에 강화하는 효과적인 방법입니다.

AI 시대 플랫폼 팀의 확장된 역할

AI의 등장은 플랫폼 팀에게 위협이 아니라, **역할 확장의 기회**입니다.

플랫폼 팀은 이제 **세 가지 축**으로 진화합니다.



Tech Enabler

기술 기반의 자율성과 표준화를 가능하게 하는 팀



표준화 민첩성 유연성



Culture Builder

협업 문화와 개발자 경험을 설계하는 팀



투명성 자율성 공동체



AI Integrator

AI를 조직의 지능으로 통합하는 전략적 두뇌



지능 예측 확장

이제 AI는 '도구'가 아니라 **조직의 두 번째 두뇌(Second Brain)**입니다.

플랫폼 팀은 이 두뇌가 올바르게 작동하도록 **정보의 흐름을 설계하는 신경망(Network)** 역할을 맡게 됩니다.

정리

Message

 DevOps -> DevEx & DevRel -> DevIntelligence

 플랫폼 팀의 수준 = 개발자들이 얼마나 행복하고 잘 협업하는가

 개발자들이 행복하게 일하는 회사가 AI 시대의 승자

마지막 질문

 "여러분의 개발자들은 행복합니까?"

Remember

 **문화**
플랫폼은 기술이 아니라 **문화** 다
사람에게 투자하라

 **활성화**
좋은 플랫폼은 **통제하지 않고 활성화** 시킨다
자율성을 보장하라

 **연결**
개발자들이 서로를 도울 수 있게 **연결** 한다
커뮤니티를 만들어라

 "여러분의 팀들은 잘 협업하고 있습니까?"

References

 **Microsoft** [Time Warp: The Gap Between Developer's Ideal vs Actual Workweeks in an AI-Driven Era](#)

 **Google** [What Predicts Software Deelopers' Productivity?](#)

 **CloudBees** [Platform Engineering is a Key Driver of Developer Productivity and Experience](#)

 **Stripe** [The Developer Coefficient](#)

 **SpringerOpen** [Why technology adoption succeeds or fails](#)